

TEACH YOURSELF LECTURE

Lecture 02

Computational Thinking & Algorithm

Chapter lecture — 13 topics with explanations, worked examples, Urdu notes, practice questions and a full answer key.

Computer Science XI · Bilingual support: English + اردو

کمپیوٹر سائنس — گیارہویں جماعت: خود سیکھیں، خود پرکھیں

13 teach-yourself lectures in this pack

★ Golden topics, Urdu notes, review and answer key

XI

How to use this lecture

Learn this chapter **on your own**. Read *Learn it*, try the worked example on paper, then answer *Check yourself* before looking at the answers. The green box repeats the idea in Urdu. ★ Golden topics are high-yield in board exams.

یہ لیکچر خود سیکھیں۔ مثال کاغذ پر حل کریں، پھر جواب دیکھنے سے پہلے خود کو پرکھیں۔

Lectures in this pack

Chapter 2: Computational Thinking & Algorithm. Study one lecture at a time, then use the chapter review and answer key at the end of this PDF.

L01. 2.1 Computational Thinking

L03. 2.2.2 Algorithm vs Pseudocode

L05. 2.3.2 Pattern Recognition

L07. 2.4 Sorting and Searching Algorithms

L09. 2.4.1 Sorting Algorithms: Selection Sort

L11. 2.4.2 Searching Algorithms: Binary Search

L13. 2.6 How to Decide Which Algorithm to Use

L02. 2.2 Algorithm

L04. 2.3.1 Decomposition

L06. 2.3.3 Abstraction

L08. 2.4.1 Sorting Algorithms: Bubble Sort

L10. 2.4.2 Searching Algorithms: Linear Search

L12. 2.5 Evaluation Criteria of Algorithm

CHAPTER 2

Computational Thinking & Algorithm

This chapter teaches you how to think like a computer scientist: break a problem into parts, spot patterns, ignore what does not matter, and write clear step-by-step algorithms. You will then learn four famous algorithms — Bubble Sort, Selection Sort, Linear Search and Binary Search — and how to judge and choose between them.

اس باب میں آپ سیکھیں گے کہ کسی مسئلے کو کمپیوٹر سائنس دان کی طرح کیسے سوچ کر حل کیا جاتا ہے: مسئلے کو چھوٹے حصوں میں توڑنا، پیٹرن پہچاننا، غیر ضروری باتوں کو نظر انداز کرنا اور مرحلہ وار الگورتھم لکھنا۔ اس کے بعد آپ ترتیب دینے (Sorting) اور تلاش کرنے (Searching) کے چار اہم الگورتھم سیکھیں گے۔

What you will learn

- Define computational thinking and name its four parts: decomposition, pattern recognition, abstraction and algorithms.
- Compare an algorithm with pseudocode on at least five points.
- Dry-run Bubble Sort and Selection Sort pass by pass on a given list, and write them in pseudocode and Python.
- List the advantages and disadvantages of each sorting and searching algorithm.
- Evaluate an algorithm for correctness (using a trace table), efficiency and clarity.
- Define an algorithm and state five reasons why algorithms are important.
- Apply decomposition, pattern recognition and abstraction to real-life problems.
- Trace Linear Search step by step and Binary Search using a low / mid / high table.
- State the best-case and worst-case time complexity of the four algorithms.
- Choose the right algorithm for a situation and justify your choice.

2.1 Computational Thinking

LEARN IT

Computational Thinking (CT) is a systematic approach to solving problems. "Systematic" means you follow a fixed, organised method instead of guessing. CT involves four activities:

- **Decomposition** — breaking a big problem down into smaller parts.
- **Pattern recognition** — identifying things that repeat or follow a rule.
- **Abstraction** — hiding (ignoring) details that do not matter.
- **Algorithms** — developing clear step-by-step instructions to solve the problem.

CT is not "thinking like a computer". It is thinking in a way that lets *you* describe a solution so clearly that a computer (or another person) could carry it out.



You already use computational thinking in daily life. When you plan your exam preparation, you split the syllabus into chapters (decomposition), notice that every chapter needs the same routine of reading, making notes and practising (pattern recognition), ignore things that are not in the syllabus (abstraction), and make a day-by-day timetable (algorithm).

LEARN IT

Did you know? Pseudocode is a human-readable representation of an algorithm using plain English and structured programming keywords. It is the bridge between human logic and machine code. You will study it in detail in section 2.2.2.

REMEMBER IT

Remember the four parts of CT with the phrase **"Don't Pick Any Apple"**: Decomposition, Pattern recognition, Abstraction, Algorithm.

COMMON MISTAKE

Students sometimes think computational thinking means "writing code". It does not. CT happens *before* coding. You can do all four steps with pen and paper, without a computer.

اردو میں سمجھیں

کمپیوٹیشنل تھنکنگ (Computational Thinking) مسائل حل کرنے کا ایک منظم طریقہ ہے۔ اس کے چار حصے ہیں: مسئلے کو چھوٹے حصوں میں توڑنا (Decomposition)، دہرائی جانے والی باتوں کو پہچاننا (Pattern Recognition)، غیر ضروری تفصیلات کو چھپانا (Abstraction) اور مرحلہ وار حل لکھنا (Algorithm)۔ یہ سوچنے کا طریقہ ہے، کوڈ لکھنا نہیں۔

CHECK YOURSELF

1. Name the four parts of computational thinking.
2. Is computational thinking done before or after writing code?
3. What is pseudocode called a "bridge" between?

Answers: 1. Decomposition, pattern recognition, abstraction, algorithms. 2. Before coding (it is planning the solution). 3. Between human logic and machine code.

2.2 Algorithm ★ Golden

LEARN IT

An **algorithm** is a step-by-step logical procedure used to solve a problem. It is **conceptual** in nature (it is an idea, a plan) and completely **independent of any programming language**. The same algorithm can be written later in Python, C++ or Java.

A good algorithm has:

- **Input** — the data it starts with.
- **Clear steps** — each step has only one meaning.
- **A definite end** — it must finish after a limited number of steps.
- **Output** — the result it produces.

WORKED EXAMPLE

Algorithm to find the larger of two numbers

Step 1: Start

Step 2: Read A and B

Step 3: If $A > B$, print "A is larger"

Step 4: Else print "B is larger or equal"

Step 5: End

Notice: there is no Python or C++ here. Anyone — even a non-programmer — can follow these steps. That is what "language-independent" means.

2.2.1 Importance of Algorithms

Reason	What it means for you
Structured approach	Algorithms provide a structured approach to problem-solving, making complex tasks manageable.
Optimisation	They optimise time and memory usage, which is crucial for large-scale applications (for example, searching millions of records).
Reusability	They can be reused for similar problems, saving time and effort. A sorting algorithm for marks can also sort prices.
Accuracy and consistency	They ensure accuracy and consistency, reducing human error. The same steps always give the same result.
Blueprint	They act as a blueprint (a plan) before actual code implementation, just like an architect's map before building a house.

REMEMBER IT

Remember the five reasons as **S-O-R-A-B**: **S**tructured, **O**ptimised, **R**eusable, **A**ccurate, **B**lueprint.

COMMON MISTAKE

Do not write "an algorithm is a program". A program is the algorithm *written in a programming language* so that a computer can run it. The algorithm itself is only the logic.

الگورتھم (Algorithm) کسی مسئلے کو حل کرنے کا مرحلہ وار منطقی طریقہ ہے۔ یہ ایک تصور ہے اور کسی بھی پروگرامنگ زبان پر منحصر نہیں۔ الگورتھم کام کو منظم بناتے ہیں، وقت اور میموری بچاتے ہیں، دوبارہ استعمال ہو سکتے ہیں، غلطیاں کم کرتے ہیں اور کوڈ لکھنے سے پہلے نقشے کا کام دیتے ہیں۔

EXAM TIP

Very common short question: "Define algorithm and write its importance." Write the definition word for word — *a step-by-step logical procedure used to solve a problem; conceptual and independent of any programming language* — then list at least four of the five importance points. If asked to "write an algorithm", always begin with **Step 1: Start** and finish with **End**, and number every step.

CHECK YOURSELF

1. Why is an algorithm called "language-independent"?
2. Give two reasons why algorithms are important.
3. Write a 5-step algorithm to find the sum of two numbers.

Answers: 1. Because it describes only the logic; it is not tied to Python, C++ or any other language. 2. Any two: structured approach, optimise time/memory, reusable, accurate and consistent, blueprint before coding. 3. Step 1: Start; Step 2: Read A and B; Step 3: SUM = A + B; Step 4: Print SUM; Step 5: End.

2.2.2 Algorithm vs Pseudocode ★ Golden

LEARN IT

Both algorithms and pseudocode are used in problem-solving, but they serve different purposes in the design phase. The **algorithm** says *what* must be done. The **pseudocode** shows *how* it could be implemented, using programming-like keywords such as IF, FOR and WHILE, but it still cannot be run on a computer.

Point	Algorithm	Pseudocode
Definition	A step-by-step logical procedure to solve a problem.	A human-readable representation of an algorithm using plain English & keywords.
Nature	Conceptual and language-independent.	Semi-formal and structured, but not executable code.
Purpose	To define what needs to be done logically.	To present how it can be implemented in a readable format.
Form	Usually described in plain steps, bullets or numbered points.	Written using structured constructs (IF, FOR, WHILE) like programming.
Focus	Focuses on the logic and steps of the problem.	Focuses on representing steps clearly before coding.
Audience	For anyone trying to understand the logic (including non-programmers).	Mainly for programmers or students preparing for coding.
Format rules	No fixed format; can be informal.	Follows standard pseudocode conventions for clarity.

WORKED EXAMPLE

The same problem written both ways: print whether a number is even or odd.

Algorithm

Step 1: Start
Step 2: Read a number N
Step 3: Divide N by 2 and find the remainder
Step 4: If the remainder is 0, print "Even",
otherwise print "Odd"
Step 5: End

Pseudocode

```
START
INPUT N
SET R TO N MOD 2
IF R = 0 THEN
    PRINT "Even"
ELSE
    PRINT "Odd"
END IF
END
```

The equivalent Python program (real, executable code):

```
n = 7
if n % 2 == 0:
    print("Even")
else:
    print("Odd")
```

OUTPUT

Odd

Problem → Algorithm (what) → Pseudocode (how) → Program (code) → Output

REMEMBER IT

Teacher's analogy: an **algorithm** is like a recipe ("Bake a cake at 350 degrees"). **Pseudocode** is like a structured kitchen manual using standard verbs (SET temperature TO 350; WHILE cake is not baked, WAIT). The program is the oven actually doing it.

COMMON MISTAKE

Pseudocode is **not** executable. If you type pseudocode into Python, it will give errors. Also, do not mix exact Python syntax (colons, brackets) into pseudocode answers unless asked — use keywords like IF ... THEN ... END IF.

اردو میں سمجھیں

الگورتھم بتاتا ہے کہ کیا کرنا ہے، جبکہ سوڈو کوڈ (Pseudocode) بتاتا ہے کہ اسے کیسے کیا جا سکتا ہے۔ الگورتھم سادہ مرحلوں میں لکھا جاتا ہے اور ہر کوئی اسے سمجھ سکتا ہے۔ سوڈو کوڈ میں IF، FOR، WHILE جیسے الفاظ استعمال ہوتے ہیں اور یہ زیادہ تر پروگرامرز کے لیے ہوتا ہے، لیکن اسے کمپیوٹر پر چلایا نہیں جا سکتا۔

EXAM TIP

"Differentiate between algorithm and pseudocode" is a favourite 4–5 mark question. Draw a two-column table and give at least **five** points (definition, nature, purpose, form, audience). Adding the recipe vs kitchen-manual analogy and one small example of each earns full marks.

CHECK YOURSELF

1. Which one — algorithm or pseudocode — uses keywords like FOR and WHILE?
2. Which one is meant for non-programmers too?
3. Can pseudocode be executed by a computer?

Answers: 1. Pseudocode. 2. Algorithm. 3. No; it is semi-formal and not executable.

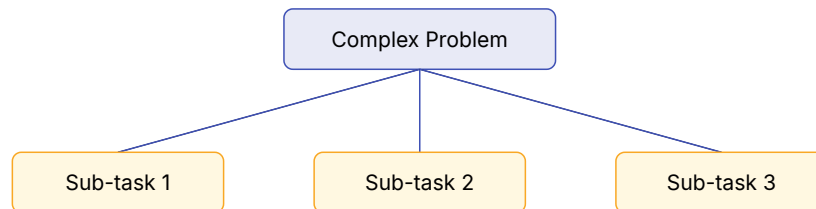
2.3.1 Decomposition ★ Golden

2.3 Algorithmic Strategies for Problem Solving. An **algorithmic strategy** is a general method that helps you design an algorithm. The three strategies in your syllabus are decomposition, pattern recognition and abstraction.

LEARN IT

Decomposition is a problem-solving technique in which a complex task or system is broken down into smaller, manageable parts. By dividing a big problem into simpler steps, it becomes easier to understand, develop and solve.

Each small part is called a **sub-task** (or sub-problem). You solve each sub-task separately, then join the solutions to solve the whole problem.



Decomposition: one complex problem is split into smaller sub-tasks.

WORKED EXAMPLE

Real-life example: Learning a new language

"Learn English" (or any language) is too big to do in one go. Decompose it:

Sub-task	Description
Learn vocabulary	Learn the words for people, actions and things (e.g., "I", "eat", "apple").
Grammar rules	Learn the sentence structure: Subject → Verb → Object.
Learn tenses	Present, past, future (e.g., "I eat" vs. "I ate").
Sentence building	Combine words using the rules to create valid sentences: "I ate an apple."

Vocabulary → Grammar rules → Tenses → Sentence building

WORKED EXAMPLE

Computing example: a "Student Result" program

Problem: read the marks of a student in 5 subjects and print the total, percentage and grade.

- Sub-task 1: Input marks of 5 subjects.
- Sub-task 2: Calculate total = sum of the 5 marks.
- Sub-task 3: Calculate percentage = $\text{total} \div 500 \times 100$.
- Sub-task 4: Decide the grade from the percentage.
- Sub-task 5: Print the result card.

Each sub-task is now so small that you can write it in two or three lines of code.

Why is decomposition useful?

- **Reduces complexity:** you focus on one piece at a time.
- **Helps in debugging:** you can find exactly where the mistake is (e.g., in the language example, is the error in the verb tense or the word order?).
- **Promotes reusability:** once you learn or build a small part, you can apply it elsewhere (the "calculate percentage" part can be reused in an attendance program).

REMEMBER IT

Think of eating a big *roti*: you cannot eat it in one bite, so you break it into small pieces. Decomposition = "break the roti".

COMMON MISTAKE

Do not break a problem into random pieces. Each sub-task must be a meaningful, clear job, and all sub-tasks together must solve the *whole* problem. If one part is missing (e.g., "print result"), the solution is incomplete.

اردو میں سمجھیں

ڈیکمپوزیشن (Decomposition) کا مطلب ہے کسی بڑے اور پیچیدہ مسئلے کو چھوٹے اور آسان حصوں میں تقسیم کرنا۔ مثلاً نئی زبان سیکھنے کو الفاظ، گرامر، زمانے (Tenses) اور جملہ سازی میں تقسیم کیا جا سکتا ہے۔ اس سے پیچیدگی کم ہوتی ہے، غلطی ڈھونڈنا آسان ہوتا ہے اور حصے دوبارہ استعمال ہو سکتے ہیں۔

EXAM TIP

Expect "Define decomposition with a real-life example" or "Explain the benefits of decomposition". Give the definition, draw the small tree (Complex Problem → Sub-task 1, 2, 3), show the language-learning table, and list the three benefits: reduces complexity, helps debugging, promotes reusability.

CHECK YOURSELF

1. What is each smaller part of a decomposed problem called?
2. Decompose "Making a school timetable" into three sub-tasks.
3. State the three benefits of decomposition.

Answers: 1. A sub-task (sub-problem). 2. Example: list subjects and teachers; decide periods per subject per week; place periods in days so no teacher has two classes at once. 3. Reduces complexity, helps in debugging, promotes reusability.

2.3.2 Pattern Recognition

LEARN IT

Pattern recognition is the process of identifying regularities, repetitions or rules in data. It helps in reducing the overall complexity of an algorithm and forms the foundation of concepts such as **loops**.

How it works — ask yourself three questions:

- What is repeating?
- What changes in a predictable way?
- Can I generalise this (write one rule that works for every case)?

WORKED EXAMPLE

Example: Triangle of stars

OUTPUT

```
*  
**  
***  
****  
*****
```

Step 1 — What repeats? Every row prints stars.

Step 2 — What changes predictably? Each row adds 1 star to the previous row's count.

Row	Stars
1	1
2	2
3	3
...	...
N	N

Step 3 — Generalise: Row N = N stars. Recognising this pattern lets us write one generic loop instead of 5 separate print statements.

Without the pattern (5 statements)

```
print("*")  
print("**")  
print("***")  
print("****")  
print("*****")
```

With the pattern (one loop)

```
n = 5  
for row in range(1, n + 1):  
    print("*" * row)
```

Both programs print exactly the same triangle. But if you want 100 rows, the loop version needs only one change ($n = 100$), while the first version needs 100 lines.

REMEMBER IT

Whenever you see "the same thing again and again, with a small change each time", think **loop**. Pattern recognition is the habit of spotting that.

COMMON MISTAKE

Do not generalise from too few cases. Check your rule on several rows (or items) before trusting it. "Row N = N stars" is correct here; a different triangle (1, 3, 5, 7 stars) follows the rule "Row N = 2N - 1 stars".

اردو میں سمجھیں

پیٹرن ریکگنیشن (Pattern Recognition) کا مطلب ہے ڈیٹا میں دہرائی جانے والی چیزوں یا اصولوں کو پہچاننا۔ ستاروں کی تکیوں میں ہر قطار میں پچھلی قطار سے ایک ستارہ زیادہ ہے، یعنی قطار N میں N ستارے۔ یہ پیٹرن پہچان کر ہم پانچ الگ پرنٹ کے بجائے ایک لوپ (Loop) لکھ سکتے ہیں۔

CHECK YOURSELF

1. Which programming concept is based on pattern recognition?
2. How many stars will row 8 of the triangle have?
3. Find the rule: 2, 4, 6, 8, ...

Answers: 1. Loops. 2. 8 stars (Row N = N stars). 3. Term N = $2 \times N$ (each term adds 2).

2.3.3 Abstraction

LEARN IT

Abstraction means hiding unnecessary features and focusing only on the essential ones needed to solve a problem. It reduces complexity by ignoring what is not important (the **noise**).

When you apply abstraction you sort details into three groups:

- **Necessary steps** — the core logic that never changes.
- **Changes based on the situation** — these become **variables** (inputs) in the algorithm.
- **Abstraction items (noise)** — details you can ignore completely.

WORKED EXAMPLE

Example: applying abstraction to "Making Tea"

To make an algorithm for tea, we separate the necessary steps from the situational changes and the noise.

1. Necessary steps (core logic)	2. Changes based on situation (variables)	3. Abstraction items (ignore / noise)
1. Boil water 2. Add tea leaves/bag 3. Steep for a few minutes 4. Pour into cup 5. Serve	Ingredients (black, green, milk) Additions (sugar, lemon) Tools (stove vs kettle)	Brand of kettle Cup colour or design Where tea is made (kitchen, office)

The core abstracted logic, independent of cup colour or kettle brand:



This one algorithm now works for green tea in an office kettle and for milk tea on a kitchen stove. The middle column simply becomes inputs: `tea_type`, `addition`, `tool`.

Computers use abstraction everywhere. When you call `print("Hello")` in Python, you do not think about how the screen lights up each pixel — those details are hidden from you. A map of Karachi shows roads and landmarks but hides every tree and car: it is an abstraction of the city.

REMEMBER IT

Abstraction = "**Keep the skeleton, drop the decoration.**" Ask: "If I remove this detail, does the solution still work?" If yes, it is noise.

COMMON MISTAKE

Do not confuse abstraction with decomposition. Decomposition *splits* a problem into parts. Abstraction *removes* unimportant details. Also, do not throw away details that really matter — "boil water" can never be ignored, but kettle brand can.

ایبسٹریکشن (Abstraction) کا مطلب ہے غیر ضروری تفصیلات کو چھپا کر صرف ضروری باتوں پر توجہ دینا۔ چائے بنانے کی مثال میں پانی ابالنا، پتی ڈالنا، دم دینا، کپ میں ڈالنا اور پیش کرنا ضروری مراحل ہیں؛ چائے کی قسم یا چینی متغیرات (Variables) ہیں؛ جبکہ کیتلی کا برانڈ یا کپ کا رنگ فالتو باتیں (Noise) ہیں جنہیں نظر انداز کیا جاتا ہے۔

CHECK YOURSELF

1. What is "noise" in abstraction?
2. In the tea example, is "sugar or lemon" a necessary step, a variable, or noise?
3. How is abstraction different from decomposition?

Answers: 1. Unnecessary details that do not affect the solution. 2. A variable (it changes with the situation). 3. Decomposition breaks a problem into parts; abstraction hides details that are not important.

2.4 Sorting and Searching Algorithms

LEARN IT

Sorting and searching are both fundamental techniques of computer science. They work on **lists of the same data type** (for example, a list of all integers or a list of all names).

Sorting	Searching
Arranging a collection of items in an order. The order may be ascending (0–9, A–Z) or descending (9–0, Z–A). Used to organise data.	Locating a specific element within a list to find its position or check its existence (is it there or not?).
Examples: Bubble Sort, Selection Sort	Examples: Linear Search, Binary Search
Real life: merit list arranged from highest to lowest marks.	Real life: finding your roll number in the merit list.

Notation you will see

- **DATA** — the list (array). `DATA[1]` is the 1st element in the textbook algorithms (counting starts at 1).
- In **pseudocode and Python** counting starts at **0**: `DATA[0]` is the 1st element. This is called an **index**.
- **N** — the number of elements in the list.
- **PTR** (pointer) — a variable that holds the current position.
- **temp** — a temporary variable used to **swap** (exchange) two values.
- **Pass** — one complete run of the inner loop over the list.

WORKED EXAMPLE

How a swap works with temp. Suppose $A = 8$ and $B = 4$. To exchange them:

`temp = A (temp = 8)` → `A = B (A = 4)` → `B = temp (B = 8)`

Without `temp`, writing `A = B` first would destroy the 8. In Python you may also write `a, b = b, a`.

COMMON MISTAKE

Position 1 in the textbook algorithm is index 0 in Python. In exams, if a question uses `DATA[1]`, count from 1; if it uses Python, count from 0. State clearly which one you use.

سارٹنگ (Sorting) کا مطلب ہے ڈیٹا کو صعودی (چھوٹے سے بڑے) یا نزولی (بڑے سے چھوٹے) ترتیب میں رکھنا۔ سرچنگ (Searching) کا مطلب ہے فہرست میں کسی خاص چیز کو تلاش کرنا تاکہ اس کی جگہ یا موجودگی معلوم ہو سکے۔ دونوں ایک ہی قسم کے ڈیٹا کی فہرست پر کام کرتے ہیں۔

CHECK YOURSELF

1. Arrange C, A, D, B in descending order.
2. What is the purpose of the variable temp?

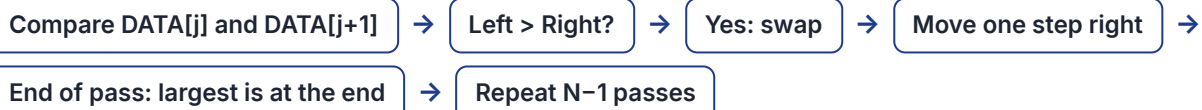
Answers: 1. D, C, B, A. 2. It temporarily holds one value so that two values can be swapped without losing either.

2.4.1 Sorting Algorithms: Bubble Sort ★ Golden

LEARN IT

Bubble Sort is an elementary sorting algorithm that works by repeatedly comparing **adjacent** (side-by-side) elements and swapping them if they are in the wrong order.

In each pass, the largest remaining value "bubbles up" to the end of the list, like an air bubble rising to the top of water. So after pass 1 the largest value is in the last place, after pass 2 the second largest is in second-last place, and so on. A list of N items needs $N - 1$ passes, and pass K makes $N - K$ comparisons.



Algorithm (textbook)

```

Step 1: Start
Step 2: Set DATA := {8,4,1,9,3}
Step 3: Set N := 5
Step 4: Repeat Step 5 and 6 for
        K = 1 to N-1
Step 5: Set PTR := 1
Step 6: Repeat while PTR ≤ N-K:
    a: If DATA[PTR] > DATA[PTR+1]:
        1. Set temp = DATA[PTR]
        2. Set DATA[PTR] = DATA[PTR+1]
        3. Set DATA[PTR+1] = temp
    b: Set PTR := PTR + 1
Step 7: End
  
```

Pseudocode

```

Start
DATA = {8,4,1,9,3}
N = 5
For outer = 0 to N-1
    For inner = 0 to N-2-outer
        IF DATA[inner] > DATA[inner+1]
            Temp = DATA[inner]
            DATA[inner] = DATA[inner+1]
            DATA[inner+1] = Temp
        End If
    End For
End For
End
  
```

WORKED EXAMPLE

Worked example: sort 8, 4, 1, 9, 3 in ascending order. The pair in brackets [] is being compared. Positions are counted from 1.

Pass 1 (K = 1, 4 comparisons)

Comparison	Pair	Left > Right?	Action	List after
1st vs 2nd	[8, 4], 1, 9, 3	8 > 4 Yes	Swap	4, 8, 1, 9, 3
2nd vs 3rd	4, [8, 1], 9, 3	8 > 1 Yes	Swap	4, 1, 8, 9, 3
3rd vs 4th	4, 1, [8, 9], 3	8 > 9 No	No change	4, 1, 8, 9, 3
4th vs 5th	4, 1, 8, [9, 3]	9 > 3 Yes	Swap	4, 1, 8, 3, 9

9 is now at its correct (final) place.

Pass 2 (K = 2, 3 comparisons)

Comparison	Pair	Left > Right?	Action	List after
1st vs 2nd	[4, 1], 8, 3, 9	4 > 1 Yes	Swap	1, 4, 8, 3, 9
2nd vs 3rd	1, [4, 8], 3, 9	4 > 8 No	No change	1, 4, 8, 3, 9
3rd vs 4th	1, 4, [8, 3], 9	8 > 3 Yes	Swap	1, 4, 3, 8, 9

Pass 3 (K = 3, 2 comparisons)

Comparison	Pair	Left > Right?	Action	List after
1st vs 2nd	[1, 4], 3, 8, 9	1 > 4 No	No change	1, 4, 3, 8, 9
2nd vs 3rd	1, [4, 3], 8, 9	4 > 3 Yes	Swap	1, 3, 4, 8, 9

Pass 4 (K = 4, 1 comparison)

Comparison	Pair	Left > Right?	Action	List after
1st vs 2nd	[1, 3], 4, 8, 9	1 > 3 No	No change	1, 3, 4, 8, 9

Sorted list: 1, 3, 4, 8, 9. Totals: $4 + 3 + 2 + 1 = 10$ comparisons and **6 swaps**.

Pass	List after pass	Fixed at the end
Start	8, 4, 1, 9, 3	—
1	4, 1, 8, 3, 9	9
2	1, 4, 3, 8, 9	8, 9
3	1, 3, 4, 8, 9	4, 8, 9
4	1, 3, 4, 8, 9	all

WORKED EXAMPLE

Bubble Sort in Python (follows the pseudocode exactly; Python indexes start at 0):

```
data = [8, 4, 1, 9, 3]
n = len(data)
for outer in range(n - 1):
    for inner in range(n - 1 - outer):
        if data[inner] > data[inner + 1]:
            temp = data[inner]
            data[inner] = data[inner + 1]
            data[inner + 1] = temp
    print("After pass", outer + 1, ":", data)
print("Sorted list:", data)
```

OUTPUT

```
After pass 1 : [4, 1, 8, 3, 9]
After pass 2 : [1, 4, 3, 8, 9]
After pass 3 : [1, 3, 4, 8, 9]
After pass 4 : [1, 3, 4, 8, 9]
Sorted list: [1, 3, 4, 8, 9]
```

`range(n - 1 - outer)` gives `inner = 0 ... N-2-outer`, exactly as in the pseudocode. Notice the list was already sorted after pass 3, but the plain algorithm still does pass 4.

Advantages and disadvantages

Advantages	Disadvantages
Easiest algorithm to visualise and explain.	Uses nested loops, so the time grows very fast as the list grows (about N^2 comparisons).
Does not need extra memory (in-place sorting).	Performs a massive number of swaps.
Stable (equal items stay in their original order).	Useless for real-world "Big Data".

REMEMBER IT

Bubble = neighbours. Bubble Sort only ever compares *neighbours* (j and $j+1$). Each pass pushes the biggest bubble to the end. For **descending** order, just change `>` to `<` in the comparison.

COMMON MISTAKE

The slides say the time grows "exponentially". Strictly, Bubble Sort's time grows **quadratically**: for N items it makes $N(N-1)/2$ comparisons, which is $O(N^2)$. Doubling the list makes it about 4 times slower — bad, but not exponential. Another common mistake: students forget that pass K does only $N - K$ comparisons; the sorted tail is never compared again.

اردو میں سمجھیں

بل سارٹ (Bubble Sort) میں ساتھ ساتھ موجود دو عناصر کا موازنہ کیا جاتا ہے اور اگر وہ غلط ترتیب میں ہوں تو ان کی جگہ بدل دی جاتی ہے۔ ہر پاس (Pass) کے بعد سب سے بڑا عدد آخر میں پہنچ جاتا ہے۔ N عناصر کے لیے $N-1$ پاس درکار ہوتے ہیں۔ یہ سمجھنے میں آسان اور مستحکم (Stable) ہے لیکن بڑے ڈیٹا کے لیے بہت سست ہے۔

EXAM TIP

The most common question: "Sort the list ... using Bubble Sort. Show each pass." Draw one table per pass with columns *Pair*, *Compare*, *Action*, *List after*. Mark the element fixed at the end after every pass. Also be ready to write the algorithm (with PTR and K) or the pseudocode (with outer/inner), and to list two advantages and two disadvantages.

CHECK YOURSELF

1. How many passes and comparisons does Bubble Sort need for 6 elements?
2. Show the list 5, 1, 4 after pass 1 of Bubble Sort.
3. Sort 2, 9, 4, 7 in descending order with Bubble Sort: give the list after each pass.

Answers: 1. 5 passes; $5 + 4 + 3 + 2 + 1 = 15$ comparisons. 2. (5,1) swap $\rightarrow$ 1, 5, 4; (5,4) swap $\rightarrow$ 1, 4, 5. 3. Pass 1: 9, 4, 7, 2; Pass 2: 9, 7, 4, 2; Pass 3: 9, 7, 4, 2.

2.4.1 Sorting Algorithms: Selection Sort ★ Golden

LEARN IT

Selection Sort is a simple sorting algorithm that works by repeatedly **selecting the smallest element** from the unsorted portion of the list and moving it to its correct position at the front.

Think of it as filling seats from the front: first find who belongs in position 1, then position 2, and so on. In the textbook version, the element at position K is compared with every element after it; whenever a smaller one is found, the two are swapped. So at the end of pass K , the K -th smallest value sits in position K .



Algorithm (textbook)

```
Step 1: Start
Step 2: Set DATA := {8,4,1,9,3}
Step 3: Set N := 5
Step 4: Repeat Step 5 and 6 for
        K = 1 to N
Step 5: Set PTR := K+1
Step 6: Repeat while PTR ≤ N:
    a: If DATA[K] > DATA[PTR]:
        1. Set temp = DATA[K]
        2. Set DATA[K] = DATA[PTR]
        3. Set DATA[PTR] = temp
    b: Set PTR := PTR + 1
Step 7: End
```

Pseudocode

```
Start
DATA = {8,4,1,9,3}
N = 5
For outer = 0 to N-1
    For inner = outer+1 to N-1
        IF DATA[outer] > DATA[inner]
            Temp = DATA[outer]
            DATA[outer] = DATA[inner]
            DATA[inner] = Temp
        End If
    End For
End For
End
```

REMEMBER IT

Teacher's tip — spot the difference: in Selection Sort the inner loop starts at `outer + 1` and compares with the *fixed* position `DATA[outer]`. In Bubble Sort the inner loop always starts at `0` and compares *neighbours* `DATA[inner]` and `DATA[inner+1]`. Bubble pushes the largest to the **end**; Selection brings the smallest to the **front**.

WORKED EXAMPLE

Worked example: sort 8, 4, 1, 9, 3 in ascending order (textbook method). The two positions in brackets are being compared.

Pass 1 — find the smallest for Position 1

Compare	List	Pos1 > PosX?	Action	List after
Pos1 vs Pos2	[8], [4], 1, 9, 3	8 > 4 Yes	Swap	4, 8, 1, 9, 3
Pos1 vs Pos3	[4], 8, [1], 9, 3	4 > 1 Yes	Swap	1, 8, 4, 9, 3
Pos1 vs Pos4	[1], 8, 4, [9], 3	1 > 9 No	No change	1, 8, 4, 9, 3
Pos1 vs Pos5	[1], 8, 4, 9, [3]	1 > 3 No	No change	1 , 8, 4, 9, 3

1 is now permanently in Position 1.

Pass 2 — find the smallest for Position 2

Compare	List	Pos2 > PosX?	Action	List after
Pos2 vs Pos3	1, [8], [4], 9, 3	8 > 4 Yes	Swap	1, 4, 8, 9, 3
Pos2 vs Pos4	1, [4], 8, [9], 3	4 > 9 No	No change	1, 4, 8, 9, 3
Pos2 vs Pos5	1, [4], 8, 9, [3]	4 > 3 Yes	Swap	1, 3 , 8, 9, 4

Pass 3 — find the smallest for Position 3

Compare	List	Pos3 > PosX?	Action	List after
Pos3 vs Pos4	1, 3, [8], [9], 4	8 > 9 No	No change	1, 3, 8, 9, 4
Pos3 vs Pos5	1, 3, [8], 9, [4]	8 > 4 Yes	Swap	1, 3, 4 , 9, 8

Pass 4 — find the smallest for Position 4

Compare	List	Pos4 > PosX?	Action	List after
Pos4 vs Pos5	1, 3, 4, [9], [8]	9 > 8 Yes	Swap	1, 3, 4, 8, 9

Sorted list: 1, 3, 4, 8, 9. Totals: 4 + 3 + 2 + 1 = **10 comparisons, 6 swaps** in this textbook method.

Pass	List after pass	Fixed at the front
Start	8, 4, 1, 9, 3	—
1	1, 8, 4, 9, 3	1
2	1, 3, 8, 9, 4	1, 3
3	1, 3, 4, 9, 8	1, 3, 4
4	1, 3, 4, 8, 9	all

WORKED EXAMPLE

Selection Sort in Python (follows the textbook pseudocode):

```
data = [8, 4, 1, 9, 3]
n = len(data)
for outer in range(n - 1):
    for inner in range(outer + 1, n):
        if data[outer] > data[inner]:
            temp = data[outer]
            data[outer] = data[inner]
            data[inner] = temp
    print("After pass", outer + 1, ":", data)
print("Sorted list:", data)
```

OUTPUT

```
After pass 1 : [1, 8, 4, 9, 3]
After pass 2 : [1, 3, 8, 9, 4]
After pass 3 : [1, 3, 4, 9, 8]
After pass 4 : [1, 3, 4, 8, 9]
Sorted list: [1, 3, 4, 8, 9]
```

WORKED EXAMPLE

The "classic" Selection Sort (one swap per pass). Many books first *find the position of the smallest value* in the unsorted part, and swap only once at the end of each pass. The result is the same; only the swaps are fewer.

```
data = [8, 4, 1, 9, 3]
n = len(data)
for outer in range(n - 1):
    smallest = outer
    for inner in range(outer + 1, n):
        if data[inner] < data[smallest]:
            smallest = inner
    data[outer], data[smallest] = data[smallest], data[outer]
    print("After pass", outer + 1, ":", data)
```

OUTPUT

```
After pass 1 : [1, 4, 8, 9, 3]
After pass 2 : [1, 3, 8, 9, 4]
After pass 3 : [1, 3, 4, 9, 8]
After pass 4 : [1, 3, 4, 8, 9]
```

Here only 4 swaps are made (at most $N - 1$). This is why Selection Sort is said to perform fewer swaps than Bubble Sort. In exams, use the textbook method unless the question asks otherwise.

Advantages and disadvantages

Advantages	Disadvantages
Very easy to code.	Time grows very fast with list size (nested loops, about N^2 comparisons).
Performs fewer swaps overall compared to Bubble Sort.	Must finish every single loop — it is "blind" to an already sorted list.
Does not need extra memory.	Not stable (it might change the relative positions of equal items).

WORKED EXAMPLE

Why "not stable"? Take two equal marks, 5_a (Ali) and 5_b (Bilal), in the list $5_a, 5_b, 2$. Pass 1 compares 5_a with 5_b (no swap), then 5_a with 2 $\rightarrow$ swap $\rightarrow 2, 5_b, 5_a$. Now Bilal is before Ali, although Ali came first originally. Bubble Sort never does this, because it swaps only neighbours that are strictly out of order.

COMMON MISTAKE

In the textbook method, the comparison is always with the **fixed position** $DATA[K]$ (Pos1 in pass 1, Pos2 in pass 2), not with the neighbour. Students often mix this up with Bubble Sort and compare neighbours. Also, after a swap, the *new* value at Pos K is used for the next comparison.

اردو میں سمجھیں

سلیکشن سارٹ (Selection Sort) میں فہرست کے غیر ترتیب شدہ حصے سے سب سے چھوٹا عدد چن کر اسے سامنے اس کی درست جگہ پر رکھا جاتا ہے۔ ہر پاس میں ایک مقررہ جگہ (Position) کا موازنہ اس کے بعد والے تمام عناصر سے ہوتا ہے۔ اس کا کوڈ آسان ہے اور ببل سارٹ سے کم تبادلے (Swaps) ہوتے ہیں، لیکن یہ مستحکم (Stable) نہیں اور ترتیب شدہ فہرست پر بھی پورے لوپ چلاتا ہے۔

EXAM TIP

Typical questions: "Sort ... using Selection Sort, showing all steps", "Differentiate between Bubble Sort and Selection Sort" and "Why is Selection Sort not stable?" In the difference question mention: what is compared (neighbours vs fixed position), where the inner loop starts (0 vs outer+1), where items settle (end vs front), stability, and number of swaps.

CHECK YOURSELF

1. In Selection Sort, where does the inner loop start?
2. Using the textbook method, show 7, 2, 5 after pass 1.
3. Which of the two sorts is stable?

Answers: 1. At outer + 1 (PTR = K + 1). 2. Pos1 vs Pos2: $7 > 2$ swap $\rightarrow 2, 7, 5$; Pos1 vs Pos3: $2 > 5$ no $\rightarrow 2, 7, 5$. 3. Bubble Sort.

2.4.2 Searching Algorithms: Linear Search

LEARN IT

Linear Search, also known as **sequential search**, works by examining each element in the list one by one, from the start, until the target is found (or the list ends). It works on both **sorted and unsorted** data.

A **flag** variable (here `ValueFound`) starts at 0 and becomes 1 when the item is found. After the loop, the flag tells us whether to print the item or "Not Found".



Algorithm (textbook)

```
Step 1: Start
Step 2: DATA := {8,3,6,1,7,2,4,5}
Step 3: N := 8
Step 4: FindValue := 7
Step 5: ValueFound := 0
Step 6: Repeat for PTR = 1 to N
Step 7:   If DATA[PTR] = FindValue:
           a. ValueFound = 1
           b. Goto Step 8
Step 8: If ValueFound = 1:
           Print DATA[PTR]
       Else:
           Print "Not Found"
Step 9: End
```

Pseudocode

```
Start
DATA = {8, 3, 6, 1, 7, 2, 4, 5}
N = 8
FindValue = 7
ValueFound = 0
For PTR = 0 to N-1
    If DATA[PTR] = FindValue then
        ValueFound = 1
        Exit Loop
    End If
End For
If ValueFound = 1 then
    Print DATA[PTR]
Else
    Print "Data Not Found"
End If
End
```

WORKED EXAMPLE

Worked example. Data list: 8, 3, 6, 1, 7, 2, 4, 5. Searching item: 7.

Step	Element checked	Condition (== 7)	Result
1	8	8 == 7 ?	Not found
2	3	3 == 7 ?	Not found
3	6	6 == 7 ?	Not found
4	1	1 == 7 ?	Not found
5	7	7 == 7 ?	Found! (Exit loop)

7 is found at position 5 (index 4) after 5 comparisons. The remaining items 2, 4, 5 are never checked.

If the item is missing — e.g., search for 9 — all 8 elements are checked, the flag stays 0, and "Data Not Found" is printed. This is the **worst case**: N comparisons.

WORKED EXAMPLE

Linear Search in Python

```
data = [8, 3, 6, 1, 7, 2, 4, 5]
find_value = 7
value_found = 0
for ptr in range(len(data)):
    print("Step", ptr + 1, ": checking", data[ptr])
    if data[ptr] == find_value:
        value_found = 1
        break
if value_found == 1:
    print("Found", data[ptr], "at position", ptr + 1)
else:
    print("Data Not Found")
```

OUTPUT

```
Step 1 : checking 8
Step 2 : checking 3
Step 3 : checking 6
Step 4 : checking 1
Step 5 : checking 7
Found 7 at position 5
```

break is Python's "Exit Loop".

Advantages and disadvantages

Advantages	Disadvantages
Does not care whether data is scrambled or sorted.	If the target is at the end of 1 million items, it does 1 million checks.
Extremely simple code (one loop, one if statement).	Time grows linearly (double the list = double the time).
Faster for very small lists (no setup time, such as sorting).	Inefficient if you need to search the same list thousands of times.
Does not require any extra space.	

REMEMBER IT

Linear Search is like looking for your friend in a queue by checking every face from the front. Best case: 1 check (first person). Worst case: N checks (last person or not there).

COMMON MISTAKE

Do not print "Not Found" inside the loop after every mismatch — that would print it many times. Decide only **after** the loop ends, using the flag. And remember to **exit the loop** once the item is found, or you waste comparisons.

اردو میں سمجھیں

لینیئر سرچ (Linear Search) کو ترتیب وار تلاش (Sequential Search) بھی کہتے ہیں۔ اس میں فہرست کے ہر عنصر کو شروع سے ایک ایک کر کے چیک کیا جاتا ہے جب تک مطلوبہ چیز نہ مل جائے۔ یہ ترتیب شدہ اور غیر ترتیب شدہ دونوں قسم کے ڈیٹا پر کام کرتی ہے اور بہت سادہ ہے، لیکن بڑی فہرست میں بہت وقت لیتی ہے۔

CHECK YOURSELF

1. What is another name for Linear Search?
2. In 8, 3, 6, 1, 7, 2, 4, 5, how many comparisons are needed to find 2?
3. What is the purpose of the variable ValueFound?

Answers: 1. Sequential search. 2. 6 comparisons (2 is in position 6). 3. It is a flag: 0 means not found yet, 1 means found; it decides what to print after the loop.

2.4.2 Searching Algorithms: Binary Search ★ Golden

LEARN IT

Binary Search is a highly efficient search algorithm that works by repeatedly dividing the search interval in half.

Crucial limitation: it only works on a SORTED list.

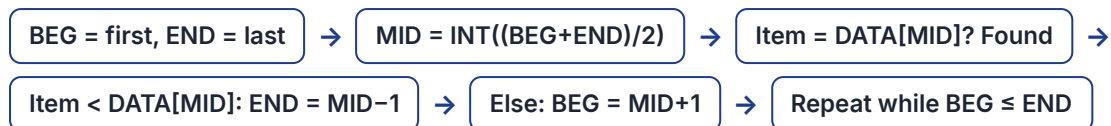
Three variables mark the part of the list still being searched:

- **BEG** (low) — first position of the current part.
- **END** (high) — last position of the current part.
- **MID** = $\text{INT}((\text{BEG} + \text{END}) / 2)$ — the middle position. INT means "drop the decimal part".

At each step compare the item with `DATA[MID]`:

- Equal → found, stop.
- Item < `DATA[MID]` → the item can only be in the **left** half: set `END = MID - 1`.
- Item > `DATA[MID]` → the item can only be in the **right** half: set `BEG = MID + 1`.

If `BEG` becomes greater than `END`, the part is empty and the item is **not in the list**.



Algorithm (textbook summary)

```

Step 1: Start
Step 2: DATA := {1,2,3,4,5,6,7,8,9}
        (Sorted)
Step 3: BEG = 1, END = 9
Step 4: MID = INT((BEG+END)/2)
Step 5: Item = 7
Step 6: LOC = 0
Step 7: Repeat while BEG ≤ END AND
        DATA[MID] ≠ Item
Step 8:   If Item < DATA[MID]:
            END = MID - 1
        Else:
            BEG = MID + 1
Step 9:   MID = INT((BEG+END)/2)
Step 10: [End loop]
Step 11: If DATA[MID] = Item:
            LOC = MID
        Else:
            LOC = 0
Step 12: Print result based on LOC
Step 13: End
  
```

Pseudocode

```

Start
DATA = {1,2,3,4,5,6,7,8,9}
BEG = 1, END = 9
MID = INT((BEG+END)/2)
Item = 7
LOC = NULL
While BEG ≤ END AND DATA[MID] ≠ Item
  If Item < DATA[MID] then
    END = MID - 1
  Else
    BEG = MID + 1
  End If
  MID = INT((BEG+END)/2)
End While
If DATA[MID] = Item then
  LOC = MID
Else
  LOC = NULL
End If
If LOC ≠ NULL then
  Print DATA[MID]
Else
  Print "Not Found"
End If
End
  
```

COMMON MISTAKE

The slide's pseudocode writes the loop condition as `BEG ≤ END OR DATA[MID] ≠ Item`. That is a slip: with OR the loop would never stop once the item is found while `BEG ≤ END`. The textbook algorithm (Step 7) correctly uses **AND** — keep going only while the part is not empty *and* the item is not yet found. Likewise, the slide sets `LOC = NULL` but then tests "`LOC ≥ 0`"; the check should simply be "`LOC is not NULL`" (or, with `LOC = 0` meaning not found, "`LOC > 0`"). The pseudocode above shows the corrected form.

WORKED EXAMPLE

Worked example 1 (textbook algorithm). DATA = 1, 2, 3, 4, 5, 6, 7, 8, 9; Item = 7. Positions 1 to 9.

Step	BEG (low)	END (high)	MID	DATA[MID]	Compare	Action
1	1	9	INT(10/2) = 5	5	7 > 5	BEG = 6 (go right)
2	6	9	INT(15/2) = 7	7	7 = 7	Found, LOC = 7

Only **2 comparisons**. Linear Search would need 7.

WORKED EXAMPLE

Worked example 2 (slide example). Sorted list: 1, 2, 3, 4, 5, 6, 7, 8. Searching item: 3.

Step	BEG (low)	END (high)	MID	DATA[MID]	Compare	Action
1	1	8	$\text{INT}(9/2) = 4$	4	$3 < 4$	END = 3 (go left)
2	1	3	$\text{INT}(4/2) = 2$	2	$3 > 2$	BEG = 3 (go right)
3	3	3	$\text{INT}(6/2) = 3$	3	$3 = 3$	FOUND at position 3

Picture the list shrinking (the highlighted cell is MID):

Step	1	2	3	4	5	6	7	8
1	1	2	3	4	5	6	7	8
2	1	2	3					
3			3					

The slide describes the same steps using the size of each sub-list: Step 1 $\text{MID} = 8/2 = 4$ (value 4), $3 < 4$, go left; Step 2 sub-list [1, 2, 3], middle = $3/2 = 1.5 \rightarrow$ 2nd item (value 2), $3 > 2$, go right; Step 3 sub-list [3], middle = 1st item (value 3), FOUND. Both ways land on the same values $4 \rightarrow 2 \rightarrow 3$.

WORKED EXAMPLE

Binary Search in Python (index starts at 0, so positions 1–8 become indexes 0–7):

```
data = [1, 2, 3, 4, 5, 6, 7, 8]
item = 3
beg = 0
end = len(data) - 1
loc = -1
step = 0
while beg ≤ end:
    mid = (beg + end) // 2
    step = step + 1
    print("Step", step, ": beg =", beg, "mid =", mid, "end =", end, "value =", data[mid])
    if data[mid] == item:
        loc = mid
        break
    elif item < data[mid]:
        end = mid - 1
    else:
        beg = mid + 1
if loc ≥ 0:
    print("Found", item, "at index", loc)
else:
    print("Not Found")
```

OUTPUT

```
Step 1 : beg = 0 mid = 3 end = 7 value = 4
Step 2 : beg = 0 mid = 1 end = 2 value = 2
Step 3 : beg = 2 mid = 2 end = 2 value = 3
Found 3 at index 2
```

// is integer division, the same as `INT()`. Index 2 is position 3. Here `loc = -1` means "not found", so the test `loc ≥ 0` is correct.

Why is it so fast?

Every comparison throws away half of the remaining list. $1,000,000 \rightarrow 500,000 \rightarrow 250,000 \rightarrow \dots \rightarrow 1$ takes about 20 halvings, because $2^{20} = 1,048,576$. So Binary Search finds an item among 1 million elements in at most about **20 steps**, while Linear Search may need 1,000,000.

Advantages and disadvantages

Advantages	Disadvantages
Extremely fast for "Big Data": finds an item in 1M elements in just ~20 steps.	The list must be sorted first; it fails on scrambled data.
Massive time savings for static lists that are searched repeatedly.	Requires random access to elements (works on arrays, not easily on linked lists).
	More complex code, prone to logic errors (off-by-one).

REMEMBER IT

Binary Search is how you find a word in a dictionary: open the middle, decide "before or after?", and throw away half the book each time. **"Sorted first, then split."**

COMMON MISTAKE

Common errors: (1) using Binary Search on an unsorted list — the answer will be wrong; (2) setting $END = MID$ instead of $MID - 1$ (or $BEG = MID$ instead of $MID + 1$), which can cause an infinite loop — this is the "off-by-one" error; (3) forgetting INT , so MID becomes a decimal like 4.5.

اردو میں سمجھیں

بائنری سرچ (Binary Search) ایک تیز رفتار طریقہ ہے جو ہر قدم پر تلاش کے حصے کو آدھا کر دیتا ہے۔ یہ صرف ترتیب شدہ (Sorted) فہرست پر کام کرتا ہے۔ درمیانی عنصر (MID) سے موازنہ کیا جاتا ہے: اگر مطلوبہ چیز چھوٹی ہو تو بائیں آدھے حصے میں، اور بڑی ہو تو دائیں آدھے حصے میں تلاش جاری رہتی ہے۔ دس لاکھ عناصر میں بھی تقریباً بیس قدموں میں نتیجہ مل جاتا ہے۔

EXAM TIP

You will usually be given a sorted list and an item and asked to "search using Binary Search, showing all steps". Always draw a table with **BEG, END, MID, DATA[MID], comparison, action**. Show the INT calculation for MID . If the item is absent, continue until $BEG > END$ and write "Not Found". Also learn: "Why must the list be sorted?" and "Compare Linear and Binary Search".

CHECK YOURSELF

1. What is the crucial condition for Binary Search to work?
2. $BEG = 6$, $END = 9$. What is MID ?
3. In 1, 2, 3, 4, 5, 6, 7, 8 (positions 1–8), list the MID values visited when searching for 8.

Answers: 1. The list must be sorted. 2. $INT(15/2) = 7$. 3. $MID = 4$ (value 4, go right, $BEG = 5$) $\rightarrow MID = INT(13/2) = 6$ (value 6, go right, $BEG = 7$) $\rightarrow MID = INT(15/2) = 7$ (value 7, go right, $BEG = 8$) $\rightarrow MID = 8$ (value 8, found). Four comparisons.

2.5 Evaluation Criteria of Algorithm

LEARN IT

Many algorithms can solve the same problem. To judge which is better, we evaluate them on three criteria: **correctness**, **efficiency** and **clarity**.

2.5.1 Correctness

An algorithm is **correct** if:

- its output matches the expected outcomes;
- it handles **edge cases** gracefully (unusual inputs such as an empty list, 0, a negative number, or an item that is not present);
- it is guaranteed to finish — no infinite loops.

We test correctness with a **trace table** (dry run): we feed sample inputs and record every value by hand, then compare the result with what we expect.

WORKED EXAMPLE

Trace table example: Even/Odd checker. Rule: divide the number by 2; if the remainder is 0 it is Even, otherwise Odd.

Input	Operation (divide by 2, find remainder)	Remainder == 0?	Result
3	3 / 2 (Quotient = 1, Remainder = 1)	No	Odd
6	6 / 2 (Quotient = 3, Remainder = 0)	Yes	Even
15	15 / 2 (Quotient = 7, Remainder = 1)	No	Odd

All three results match what we expect, so the algorithm is correct for these inputs. The same check in Python:

```
for number in [3, 6, 15]:
    remainder = number % 2
    if remainder == 0:
        print(number, "remainder", remainder, "→ Even")
    else:
        print(number, "remainder", remainder, "→ Odd")
```

OUTPUT

```
3 remainder 1 → Odd
6 remainder 0 → Even
15 remainder 1 → Odd
```

A good tester would also try the edge case 0 (remainder 0 → Even, which is correct).

2.5.2 Efficiency

We measure algorithm efficiency through **time** (speed) and **space** (memory). The key question is: how fast does it run as the input size grows — does it **scale** well?

Computer scientists describe growth with **Big O notation**. It counts roughly how many steps are needed for N items:

- **O(1)** — constant: same time, whatever N is.
- **O(log N)** — logarithmic: grows very slowly (Binary Search).
- **O(N)** — linear: double N, double the time (Linear Search).
- **O(N²)** — quadratic: double N, four times the time (Bubble and Selection Sort).

WORKED EXAMPLE

How the worst-case number of comparisons grows

N (items)	Linear Search (N)	Binary Search (about $\log_2 N$)	Bubble/Selection Sort $N(N-1)/2$
10	10	4	45
100	100	7	4,950
1,000	1,000	10	499,500
1,000,000	1,000,000	20	499,999,500,000

This is why Binary Search is called excellent for Big Data and the simple sorts are called useless for it.

2.5.3 Clarity (Readability)

An algorithm must be easy to read, understand and modify. It should use **clear steps**, **meaningful variable names** and **good indentation**.

Unclear

```
a=[3,6,15]
for x in a:
    if x%2==0:print(x,"E")
    else:print(x,"O")
```

Clear

```
numbers = [3, 6, 15]
for number in numbers:
    if number % 2 == 0:
        print(number, "Even")
    else:
        print(number, "Odd")
```

Both work, but the second is far easier for another person (or you, a month later) to understand and change.

REMEMBER IT

Remember **C-E-C**: **C**orrect (right answer, always finishes), **E**fficient (fast and light), **C**lear (easy to read).

COMMON MISTAKE

Testing only "normal" inputs is not enough to prove correctness. Always include edge cases in your trace table, e.g., the first item, the last item, a missing item, or an empty list in a search.

اردو میں سمجھیں

کسی الگورتھم کو تین معیارات پر پرکھا جاتا ہے: درستگی (Correctness) یعنی درست نتیجہ دے، غیر معمولی صورتوں کو سنبھالے اور ہمیشہ ختم ہو؛ کارکردگی (Efficiency) یعنی کم وقت اور کم میموری استعمال کرے؛ اور وضاحت (Clarity) یعنی پڑھنے، سمجھنے اور تبدیل کرنے میں آسان ہو۔ درستگی جانچنے کے لیے ٹریس ٹیبل (Trace Table) بنایا جاتا ہے۔

CHECK YOURSELF

1. Name the three evaluation criteria of an algorithm.
2. Give one example of an edge case for a search algorithm.
3. Efficiency is measured in terms of which two resources?

Answers: 1. Correctness, efficiency, clarity. 2. The item is not in the list (or the list is empty, or the item is the last element). 3. Time (speed) and space (memory).

2.6 How to Decide Which Algorithm to Use

LEARN IT

Before choosing an algorithm, ask three questions:

- **What is the input size?** (10 items or 10 million?)
- **Is the data sorted?**
- **Do we need speed or memory efficiency?**

Simple decision guide:

- If searching on **unsorted** data → use **Linear Search**.
- If searching on **sorted** data → use **Binary Search**.
- If data needs **ordering** → use a **sorting** algorithm.



Comparison of the four algorithms

Algorithm	Type	Best case	Worst case	Extra memory	Needs sorted data?	Stable?
Bubble Sort	Sorting	$O(N^2)$ as written; $O(N)$ with an early-stop check	$O(N^2)$	None — $O(1)$	—	Yes
Selection Sort	Sorting	$O(N^2)$	$O(N^2)$	None — $O(1)$	—	No
Linear Search	Searching	$O(1)$ (first item)	$O(N)$ (last item / absent)	None — $O(1)$	No	—
Binary Search	Searching	$O(1)$ (item at first MID)	$O(\log N)$	None — $O(1)$	Yes	—

Point	Bubble Sort	Selection Sort
What is compared	Adjacent elements (j and $j+1$)	Fixed position K with every later element
Inner loop starts at	0	outer + 1
After each pass	Largest moves to the end	Smallest moves to the front
Swaps	Many	Fewer
Stable	Yes	No

Point	Linear Search	Binary Search
Method	Checks one by one	Splits the list in half each step
Data required	Sorted or unsorted	Sorted only
Worst case (1M items)	1,000,000 checks	About 20 checks
Code	Very simple	More complex, off-by-one risk
Best for	Small or unsorted lists, one-off searches	Large, sorted lists searched many times

WORKED EXAMPLE

Choosing in practice

- **A shopkeeper's list of 15 items, written in random order, searched once:** Linear Search — the list is unsorted and tiny; sorting first would waste time.
- **NADRA looking up a CNIC number among millions of sorted records, thousands of times a day:** Binary Search — data is sorted, huge, and searched repeatedly.
- **A teacher arranging 40 students' marks into a merit list:** a sorting algorithm; for 40 items even Bubble or Selection Sort is acceptable.
- **An unsorted list of 1 million records that will be searched thousands of times:** sort it once, then use Binary Search for every search.

REMEMBER IT

Two keywords decide most exam answers: "**sorted?**" and "**how big?**". Unsorted or small → Linear. Sorted and big → Binary.

COMMON MISTAKE

"Binary Search is always better" is wrong. On unsorted data it does not work at all, and for a single search on a small unsorted list, sorting first costs more than a simple Linear Search.

اردو میں سمجھیں

الگورتھم چننے سے پہلے دیکھیں کہ ڈیٹا کتنا بڑا ہے، ترتیب شدہ ہے یا نہیں، اور ہمیں رفتار چاہیے یا میموری کی بچت۔ غیر ترتیب شدہ ڈیٹا میں تلاش کے لیے لینیئر سرچ، ترتیب شدہ ڈیٹا میں تلاش کے لیے بائنری سرچ، اور ڈیٹا کو ترتیب دینے کے لیے سارٹنگ الگورتھم استعمال کریں۔

CHECK YOURSELF

1. Which search would you use on an unsorted list of phone contacts?
2. What is the worst-case time complexity of Binary Search?
3. Which sort has the same time complexity in its best and worst case?

Answers: 1. Linear Search. 2. $O(\log N)$. 3. Selection Sort ($O(N^2)$ in both).

Chapter 2 Review

Chapter summary

- **Computational thinking** is systematic problem solving using decomposition, pattern recognition, abstraction and algorithms.
- An **algorithm** is a step-by-step logical procedure to solve a problem; it is conceptual and language-independent.
- Algorithms give structure, optimise time and memory, are reusable, ensure accuracy and act as a blueprint for code.
- **Pseudocode** is a human-readable, structured representation of an algorithm using keywords like IF, FOR and WHILE; it is not executable.
- An algorithm defines *what* to do; pseudocode shows *how* to implement it.
- **Decomposition** breaks a big problem into smaller sub-tasks; it reduces complexity, helps debugging and promotes reusability.
- **Pattern recognition** finds repetitions and rules (Row N = N stars) and is the foundation of loops.
- **Abstraction** hides noise and keeps only essential steps; situational details become variables.
- Sorting arranges data (ascending or descending); searching locates an element's position or existence.
- **Bubble Sort** compares and swaps adjacent elements; the largest value moves to the end each pass; N – 1 passes; stable; $O(N^2)$.
- **Selection Sort** brings the smallest remaining value to the front each pass; inner loop starts at outer + 1; fewer swaps; not stable; $O(N^2)$.
- **Linear Search** checks items one by one; works on unsorted data; $O(N)$ worst case.
- **Binary Search** halves the search range using BEG, MID and END; needs sorted data; $O(\log N)$ — about 20 steps for 1 million items.
- Algorithms are evaluated on **correctness** (checked with trace tables and edge cases), **efficiency** (time and space) and **clarity**.
- Choose by asking: input size, sorted or not, speed or memory? Unsorted → Linear; sorted → Binary; needs order → sort.

Key terms

Term	Meaning	اردو
Computational thinking	A systematic approach to solving problems using decomposition, patterns, abstraction and algorithms.	کمپیوٹیشنل سوچ
Algorithm	A step-by-step logical procedure to solve a problem, independent of any language.	الگورتھم، مرحلہ وار طریقہ حل
Pseudocode	A human-readable, structured but non-executable representation of an algorithm.	سوڈو کوڈ، فرضی کوڈ
Decomposition	Breaking a complex problem into smaller, manageable parts.	تقسیم کاری
Sub-task	One small part of a decomposed problem.	ذیلی کام
Pattern recognition	Identifying regularities, repetitions or rules in data.	نمونے کی پہچان
Abstraction	Hiding unnecessary details and focusing on essentials.	تجربہ، غیر ضروری تفصیل چھپانا
Noise	Details that do not affect the solution and can be ignored.	فالتو تفصیلات
Sorting	Arranging items in ascending or descending order.	ترتیب دینا
Searching	Locating an element in a list to find its position or existence.	تلاش کرنا
Ascending order	Smallest to largest (0–9, A–Z).	صعودی ترتیب
Descending order	Largest to smallest (9–0, Z–A).	نزولی ترتیب
Bubble Sort	Sort that repeatedly compares and swaps adjacent elements.	ببل سارٹ
Selection Sort	Sort that repeatedly selects the smallest element and puts it at the front.	سلیکشن سارٹ
Pass	One full run of the inner loop through the list.	ایک چکر
Swap	Exchanging the values of two positions, usually with a temp variable.	جگہ کا تبادلہ
In-place sorting	Sorting without needing extra memory for another list.	اضافی میموری کے بغیر ترتیب

Term	Meaning	اردو
Stable sort	A sort that keeps equal items in their original order.	مستحکم ترتیب
Linear (sequential) search	Checking each element one by one until the target is found.	ترتیب وار تلاش
Binary search	Search that halves a sorted list at each step.	دو حصی تلاش
BEG / MID / END	First, middle and last positions of the part being searched in Binary Search.	آغاز / درمیان / اختتام
Flag	A variable (like ValueFound) that records whether something has happened.	نشان دہی کرنے والا متغیر
Correctness	Giving expected output, handling edge cases and always finishing.	درستگی
Efficiency	How well an algorithm uses time and memory as input grows.	کارکردگی
Clarity	How easy an algorithm is to read, understand and modify.	وضاحت
Trace table	A table recording values step by step during a dry run.	ٹریس ٹیبل، جانچ جدول
Edge case	An unusual or extreme input used to test an algorithm.	انتہائی صورت
Time complexity (Big O)	A measure of how the number of steps grows with input size, e.g. $O(N)$, $O(\log N)$.	وقتی پیچیدگی

Practice: multiple choice

1. An algorithm is best defined as:
 - a. a programming language
 - b. a step-by-step logical procedure to solve a problem
 - c. a type of computer hardware
 - d. compiled machine code
2. Pseudocode is:
 - a. executable machine code
 - b. a symbol used in flowcharts
 - c. a Python library
 - d. a human-readable representation of an algorithm using plain English and keywords
3. Breaking a complex problem into smaller, manageable parts is called:
 - a. decomposition
 - b. abstraction
 - c. pattern recognition
 - d. compilation
4. In the "making tea" example, ignoring the brand of the kettle is an example of:
 - a. decomposition
 - b. sorting
 - c. abstraction
 - d. searching
5. Pattern recognition forms the foundation of:
 - a. variables
 - b. loops
 - c. comments
 - d. input devices
6. Bubble Sort works by comparing:
 - a. the first and last elements
 - b. every element with the middle element
 - c. adjacent elements
 - d. random pairs of elements
7. After the first pass of Bubble Sort (ascending) on 8, 4, 1, 9, 3, the list is:
 - a. 1, 3, 4, 8, 9
 - b. 4, 8, 1, 9, 3
 - c. 1, 8, 4, 9, 3
 - d. 4, 1, 8, 3, 9
8. Binary Search works only on a list that is:
 - a. sorted
 - b. unsorted
 - c. stored in a linked list
 - d. of strings only
9. Linear Search is also known as:
 - a. binary search
 - b. sequential search
 - c. bubble search
 - d. random search
10. How many passes does Bubble Sort need to sort N elements?
 - a. N
 - b. $N + 1$
 - c. $N - 1$
 - d. $2N$
11. In Selection Sort pseudocode, the inner loop starts at:
 - a. 0
 - b. $N - 1$
 - c. $\text{outer} - 1$
 - d. $\text{outer} + 1$
12. The worst-case time complexity of Binary Search is:
 - a. $O(\log N)$
 - b. $O(N)$
 - c. $O(N^2)$
 - d. $O(1)$
13. About how many steps does Binary Search need, at most, to find an item among 1 million sorted elements?
 - a. 2
 - b. 20
 - c. 1,000
 - d. 1,000,000
14. Which sorting algorithm is NOT stable?
 - a. Bubble Sort
 - b. Both are stable
 - c. Selection Sort
 - d. Neither sorts data
15. Which of the following is NOT an evaluation criterion of an algorithm?
 - a. Correctness
 - b. Efficiency
 - c. Clarity
 - d. Popularity

Practice: short questions

1. Define computational thinking and name its four components.
2. Write any four points on the importance of algorithms.
3. Give three differences between an algorithm and pseudocode.
4. What is decomposition? Explain it with the example of learning a new language.
5. What three questions should you ask during pattern recognition? Apply them to the triangle of stars.
6. Define abstraction. In the "making tea" example, name two variables and two noise items.
7. Differentiate between sorting and searching.
8. Why does Binary Search require a sorted list? State two disadvantages of Binary Search.
9. Give two differences between Bubble Sort and Selection Sort.
10. Name the three evaluation criteria of an algorithm. What is a trace table used for?

Practice: long / problem questions

1. Sort the list **5, 2, 7, 1, 6** in ascending order using Bubble Sort. Show every comparison of every pass in a table, and state the total number of comparisons and swaps. Then write the Python code for Bubble Sort.
2. Sort the list **6, 2, 9, 5, 1** in ascending order using Selection Sort (textbook method). Show every comparison of every pass and count the swaps. Also state why Selection Sort is called "blind" to an already sorted list.
3. The sorted list is **3, 8, 12, 17, 23, 31, 40, 52, 66** (Python indexes 0–8). Using Binary Search, show a low / mid / high table to search for (a) 40 and (b) 20. For each, state how many comparisons Linear Search would need.
4. Write the algorithm and pseudocode for Linear Search. Dry-run it on **8, 3, 6, 1, 7, 2, 4, 5** for FindValue = 2 and for FindValue = 9, and give two advantages and two disadvantages of Linear Search.
5. A school wants a program that reads the marks of 5 subjects for every student and prints each student's total, percentage and grade. Apply decomposition, pattern recognition and abstraction to this problem, and write a short algorithm.
6. A library has 50,000 book IDs stored in sorted order, and the librarian searches it hundreds of times a day. (a) Which search algorithm should be used and why? (b) What is the maximum number of comparisons for Linear and for Binary Search? (c) Explain the three evaluation criteria you would use to judge the chosen program.

Answer key

Multiple choice

1. b
2. d
3. a
4. c
5. b
6. c
7. d
8. a
9. b
10. c
11. d
12. a
13. b
14. c
15. d

Short questions — model answers

1. Computational thinking is a systematic approach to solving problems. Its four components are decomposition (breaking down), pattern recognition (finding repetitions), abstraction (hiding unnecessary details) and algorithms (writing step-by-step solutions).
2. Algorithms (1) provide a structured approach that makes complex tasks manageable; (2) optimise time and memory usage, crucial for large applications; (3) can be reused for similar problems; (4) ensure accuracy and consistency, reducing human error; (5) act as a blueprint before coding. (Any four.)
3. (1) An algorithm is a step-by-step logical procedure; pseudocode is a human-readable representation of it using English and keywords. (2) An algorithm is conceptual and language-independent; pseudocode is semi-formal and structured but not executable. (3) An algorithm has no fixed format and is for anyone; pseudocode uses IF/FOR/WHILE, follows conventions and is mainly for programmers.
4. Decomposition is breaking a complex task into smaller, manageable parts. Learning a language is decomposed into: vocabulary (words like "I", "eat", "apple"), grammar rules (Subject → Verb → Object), tenses ("I eat" vs "I ate") and sentence building (combining words with rules).
5. Ask: What is repeating? What changes in a predictable way? Can I generalise this? For the star triangle: stars repeat in every row; each row has one more star than the previous; generalisation: Row $N = N$ stars, so one loop can print it.
6. Abstraction means hiding unnecessary features and focusing on the essential ones. Variables: type of tea (black, green, milk), additions (sugar, lemon), tool (stove or kettle) — any two. Noise: brand of kettle, cup colour or design, where tea is made — any two.
7. Sorting arranges a collection of items in ascending or descending order to organise data (e.g., Bubble Sort, Selection Sort). Searching locates a specific element in a list to find its position or whether it exists (e.g., Linear Search, Binary Search).
8. Binary Search decides to go left or right by comparing with the middle value; this decision is only valid if all smaller values are on the left and larger on the right, i.e., the list is sorted. Disadvantages: needs sorted data; needs random access (arrays, not linked lists); code is more complex and prone to off-by-one errors. (Any two.)
9. (1) Bubble Sort compares adjacent elements; Selection Sort compares a fixed position with all later elements. (2) Bubble Sort moves the largest to the end each pass; Selection Sort moves the smallest to the front. Also: Bubble Sort is stable, Selection Sort is not; Selection Sort makes fewer swaps.
10. Correctness, efficiency (time and space) and clarity (readability). A trace table records the values of variables step by step during a dry run, so we can check that the output matches the expected result and so prove correctness.

Long questions — model answers

1. Bubble Sort on 5, 2, 7, 1, 6

Pass	Pair	Left > Right?	List after
1	[5, 2]	Yes, swap	2, 5, 7, 1, 6
1	[5, 7]	No	2, 5, 7, 1, 6
1	[7, 1]	Yes, swap	2, 5, 1, 7, 6
1	[7, 6]	Yes, swap	2, 5, 1, 6, 7
2	[2, 5]	No	2, 5, 1, 6, 7
2	[5, 1]	Yes, swap	2, 1, 5, 6, 7
2	[5, 6]	No	2, 1, 5, 6, 7
3	[2, 1]	Yes, swap	1, 2, 5, 6, 7
3	[2, 5]	No	1, 2, 5, 6, 7
4	[1, 2]	No	1, 2, 5, 6, 7

Total comparisons = 4 + 3 + 2 + 1 = 10; total swaps = 3 + 1 + 1 + 0 = 5. Sorted list: 1, 2, 5, 6, 7.

```
data = [5, 2, 7, 1, 6]
n = len(data)
for outer in range(n - 1):
    for inner in range(n - 1 - outer):
        if data[inner] > data[inner + 1]:
            temp = data[inner]
            data[inner] = data[inner + 1]
            data[inner + 1] = temp
print(data)
```

OUTPUT

```
[1, 2, 5, 6, 7]
```

2. Selection Sort (textbook method) on 6, 2, 9, 5, 1

Pass	Compare	Values	Action	List after
1	Pos1 vs Pos2	6 > 2	Swap	2, 6, 9, 5, 1
1	Pos1 vs Pos3	2 > 9	No	2, 6, 9, 5, 1
1	Pos1 vs Pos4	2 > 5	No	2, 6, 9, 5, 1
1	Pos1 vs Pos5	2 > 1	Swap	1, 6, 9, 5, 2
2	Pos2 vs Pos3	6 > 9	No	1, 6, 9, 5, 2
2	Pos2 vs Pos4	6 > 5	Swap	1, 5, 9, 6, 2
2	Pos2 vs Pos5	5 > 2	Swap	1, 2, 9, 6, 5
3	Pos3 vs Pos4	9 > 6	Swap	1, 2, 6, 9, 5
3	Pos3 vs Pos5	6 > 5	Swap	1, 2, 5, 9, 6
4	Pos4 vs Pos5	9 > 6	Swap	1, 2, 5, 6, 9

10 comparisons, 7 swaps. Sorted list: 1, 2, 5, 6, 9. (With the classic one-swap-per-pass method the passes give 1, 2, 9, 5, 6 → 1, 2, 9, 5, 6 → 1, 2, 5, 9, 6 → 1, 2, 5, 6, 9.)

Selection Sort is "blind" to a sorted list because its loops have fixed limits: it always makes all $N(N-1)/2$ comparisons, even if the list is already in order. It has no way to notice that no swaps were needed and stop early.

3. (a) Search for 40

Step	low	high	mid	data[mid]	Compare	Action
1	0	8	$(0+8)//2 = 4$	23	$40 > 23$	low = 5
2	5	8	$(5+8)//2 = 6$	40	$40 = 40$	Found at index 6

Binary: 2 comparisons. Linear Search: 40 is the 7th item, so 7 comparisons.

(b) Search for 20

Step	low	high	mid	data[mid]	Compare	Action
1	0	8	4	23	$20 < 23$	high = 3
2	0	3	$(0+3)//2 = 1$	8	$20 > 8$	low = 2
3	2	3	$(2+3)//2 = 2$	12	$20 > 12$	low = 3
4	3	3	3	17	$20 > 17$	low = 4
—	4	3	low > high, so the loop stops: Not Found			

Binary: 4 comparisons. Linear Search must check all 9 items before saying "Not Found".

4. Algorithm and pseudocode: as in section 2.4.2 (Start; DATA, N, FindValue, ValueFound := 0; repeat for PTR = 1 to N: if DATA[PTR] = FindValue then ValueFound = 1 and exit; after the loop print DATA[PTR] if ValueFound = 1, else "Not Found"; End).

FindValue = 2:

Step	1	2	3	4	5	6
Element	8	3	6	1	7	2
== 2?	No	No	No	No	No	Yes — exit

Found at position 6 after 6 comparisons; ValueFound = 1; prints 2.

FindValue = 9: 8, 3, 6, 1, 7, 2, 4, 5 are all checked (8 comparisons), none equals 9, ValueFound stays 0, so "Data Not Found" is printed.

Advantages: works on sorted and unsorted data; very simple code; fast for small lists; no extra space. Disadvantages: up to N checks (1 million for 1 million items); time grows linearly; inefficient if the same list is searched thousands of times. (Any two of each.)

5. **Decomposition:** (1) input a student's 5 marks; (2) calculate total; (3) calculate percentage = total ÷ 500 × 100; (4) decide grade; (5) print result; (6) repeat for the next student.

Pattern recognition: the same steps repeat for every student, and the same "add a mark" step repeats for every subject — so use a loop over students and a loop over subjects.

Abstraction: keep only roll number, marks, total, percentage and grade; ignore noise such as the student's address, uniform or the colour of the result card. The number of students can be a variable.

Algorithm: Step 1: Start. Step 2: Read number of students S. Step 3: Repeat Steps 4–8 for each student. Step 4: TOTAL = 0. Step 5: Repeat 5 times: read MARK, TOTAL = TOTAL + MARK. Step 6: PER = TOTAL / 500 × 100. Step 7: If PER ≥ 80 grade "A1", else if PER ≥ 70 "A", else if PER ≥ 60 "B", else if PER ≥ 50 "C", else "Fail" (school's own grading scale). Step 8: Print TOTAL, PER, GRADE. Step 9: End.

6. (a) **Binary Search**, because the data is already sorted, it is large (50,000 items) and it is searched many times a day, so the time saved per search adds up; Linear Search would be far slower.

(b) Linear Search worst case: 50,000 comparisons. Binary Search worst case: 16 comparisons, because $2^{15} = 32,768 < 50,000 \leq 2^{16} = 65,536$, so 16 halvings are enough to reduce 50,000 items to one.

(c) **Correctness:** it must return the right position for IDs that exist, say "Not Found" for missing IDs (edge cases: first ID, last ID, missing ID), and always stop. **Efficiency:** time $O(\log N)$ and no extra memory ($O(1)$). **Clarity:** meaningful names like low, mid, high, clear steps and good indentation so another programmer can maintain it.

Self-assessment: I can...

- ☐ define computational thinking and name its four parts.
- ☐ draw a table comparing algorithm and pseudocode on at least five points.
- ☐ find a pattern in data and turn it into a loop.
- ☐ dry-run Bubble Sort pass by pass and count comparisons and swaps.
- ☐ trace Linear Search with a step table, including the "not found" case.
- ☐ state best and worst time complexity for all four algorithms.
- ☐ define an algorithm and give five reasons it is important.
- ☐ decompose a real-life problem into sub-tasks and explain three benefits.
- ☐ separate necessary steps, variables and noise using abstraction.
- ☐ dry-run Selection Sort pass by pass and explain why it is not stable.
- ☐ trace Binary Search with a BEG / MID / END table and explain why it needs sorted data.
- ☐ evaluate an algorithm for correctness, efficiency and clarity, and choose the right algorithm for a situation.